

Homework 10: TLS Configuration Analysis

Nicolas Leone
Student ID: 1986354
Cybersecurity

December 21, 2025

Contents

1	Introduction	3
1.1	Assignment Requirements	3
1.2	Analyzed Servers	3
2	Methodology	3
2.1	Protocol-Level Analysis with OpenSSL	3
2.1.1	Certificate Chain Analysis	3
2.1.2	Extension Detection	4
2.2	Policy-Based Evaluation with SSL Labs	4
2.3	Comparison Approach	4
3	OpenSSL Analysis Results	5
3.1	www.google.com	5
3.1.1	Protocol Version Support	5
3.1.2	Cipher Suite Analysis	5
3.1.3	Certificate Chain	6
3.1.4	TLS Extensions	6
3.2	www.cloudflare.com	6
3.2.1	Protocol and Cipher Suite	6
3.2.2	Certificate Chain	7
3.2.3	TLS Extensions	7
3.3	github.com	7
3.3.1	Protocol and Cipher Suite	7

3.3.2	Certificate Chain	8
4	SSL Labs Analysis Results	8
4.1	Overall Scores	8
4.2	Score Interpretation	8
4.3	Vulnerability Testing	9
5	Comparative Analysis	10
5.1	Methodological Differences	10
5.2	Key Discrepancies and Implications	10
5.2.1	1. Test Coverage Difference	10
5.2.2	2. OSCP Stapling Detection	10
5.2.3	3. Forward Secrecy Granularity	11
5.2.4	4. Vulnerability Detection Gap	11
5.3	Security Posture Summary	12
5.4	Practical Implications	12
5.4.1	Tool Selection Guidelines	12
5.4.2	Configuration Recommendations	13
6	Conclusion	14
6.1	Key Findings	14
6.2	Lessons Learned	15
6.3	Final Remarks	15

1 Introduction

Transport Layer Security (TLS) is the cryptographic protocol that secures communication over computer networks, providing confidentiality, integrity, and authentication. This homework analyzes server-side TLS configurations using two approaches: protocol-level inspection with OpenSSL and policy-based evaluation with SSL Labs.

1.1 Assignment Requirements

The analysis focuses on:

- TLS version and cipher suite negotiation
- Key exchange mechanisms and forward secrecy
- Certificate chain validation
- Protocol extensions (ALPN, OCSP stapling)
- Comparison between OpenSSL and SSL Labs results

1.2 Analyzed Servers

We analyze three major web services: **www.google.com**, **www.cloudflare.com**, and **github.com**. These represent critical internet infrastructure maintained by security-conscious organizations.

2 Methodology

2.1 Protocol-Level Analysis with OpenSSL

The `openssl s_client` tool establishes TLS connections and reports detailed handshake information. The Server Name Indication (SNI) extension is essential for virtual hosting:

```
openssl s_client -connect www.google.com:443 -servername www.google.com
```

Listing 1: Basic TLS connection test

2.1.1 Certificate Chain Analysis

To extract and analyze the certificate chain:

```
# View certificate details
openssl s_client -connect server:443 -servername server < /dev/null
2>&1 | \
openssl x509 -text -noout
```

```
# Show full certificate chain
openssl s_client -connect server:443 -servername server -showcerts
```

Listing 2: Certificate inspection

2.1.2 Extension Detection

ALPN Testing:

```
openssl s_client -connect server:443 -servername server -alpn h2,
http/1.1
```

OCSP Stapling:

```
openssl s_client -connect server:443 -servername server -status
```

2.2 Policy-Based Evaluation with SSL Labs

SSL Labs (<https://www.ssllabs.com/ssltest/>) provides comprehensive assessment through its API:

```
# Initiate scan
curl "https://api.ssllabs.com/api/v3/analyze?host=www.google.com"

# Poll for results
curl "https://api.ssllabs.com/api/v3/analyze?host=www.google.com&
all=done"
```

Listing 3: SSL Labs API workflow

The service evaluates:

- **Certificate:** Chain validity, key strength (RSA $\geq$ 2048-bit, ECC $\geq$ 256-bit), signature algorithm (SHA-256+)
- **Protocol Support:** Tests TLS 1.3, 1.2, 1.1, 1.0, SSL 3.0
- **Key Exchange:** Forward secrecy support, DH parameter strength
- **Cipher Strength:** Symmetric encryption algorithm security (AES, ChaCha20)
- **Vulnerabilities:** BEAST, POODLE, Heartbleed, FREAK, Logjam, DROWN, etc.

2.3 Comparison Approach

We compare direct protocol observation (OpenSSL) with policy-based evaluation (SSL Labs) across multiple dimensions:

- **Scope:** Single connection vs. comprehensive testing

- **Depth:** Negotiated parameters vs. all supported options
- **Interpretation:** Raw data vs. policy-based grading

3 OpenSSL Analysis Results

3.1 www.google.com

3.1.1 Protocol Version Support

Version	Support	Notes
TLS 1.3	Supported	Latest protocol, mandatory forward secrecy
TLS 1.2	Supported	Backward compatibility for legacy clients
TLS 1.1	Not supported	Deprecated (RFC 8996, March 2021)
TLS 1.0	Not supported	Deprecated, vulnerable to BEAST attack

Table 1: TLS version support for www.google.com

Google's configuration follows industry best practices by disabling all deprecated protocols.

3.1.2 Cipher Suite Analysis

Connection output:

```
Protocol : TLSv1.3
Cipher   : TLS_AES_256_GCM_SHA384
```

Cipher Suite Components:

- **TLS_AES_256_GCM_SHA384:** TLS 1.3 cipher suite
 - **AES-256:** 256-bit symmetric encryption (14 rounds)
 - **GCM:** Galois/Counter Mode - authenticated encryption (AEAD)
 - **SHA384:** Hash function for HKDF key derivation
- **Forward Secrecy:** TLS 1.3 mandates ephemeral key exchange
- **Key Exchange:** X25519 (Curve25519 ECDH) - 128-bit security level

Security Assessment: This represents best-in-class cryptography. AES-256-GCM provides both confidentiality and integrity through authenticated encryption, preventing padding oracle and other attacks that affected earlier cipher modes.

3.1.3 Certificate Chain

```
Certificate chain
0 s:CN=www.google.com
  i:C=US, O=Google Trust Services, CN=WR2
1 s:C=US, O=Google Trust Services, CN=WR2
  i:C=US, O=Google Trust Services LLC, CN=GTS Root R1
2 s:C=US, O=Google Trust Services LLC, CN=GTS Root R1
  i:C=US, O=GlobalSign, OU=GlobalSign Root CA - R1

Verify return code: 0 (ok)
```

Chain Structure Analysis:

1. **End-Entity Certificate:** CN=www.google.com
 - Subject Alternative Names (SANs) for multiple domains
 - RSA 2048-bit or ECDSA P-256 public key
 - 90-day validity period (short-lived certificate)
2. **Intermediate CA:** Google Trust Services WR2
3. **Intermediate CA:** GTS Root R1
4. **Root CA:** GlobalSign Root CA - R1 (trusted by all major browsers)

Google operates its own Certificate Authority (Google Trust Services) for faster issuance and better lifecycle management.

3.1.4 TLS Extensions

Extension	Status	Purpose
SNI		Virtual hosting support
ALPN	h2	HTTP/2 negotiation
OCSP Stapling	Unclear	Certificate revocation check

Table 2: TLS extensions for www.google.com

ALPN Negotiation Result:

```
ALPN protocol: h2
```

HTTP/2 support enables multiplexing, header compression (HPACK), and server push for improved performance.

3.2 www.cloudflare.com

3.2.1 Protocol and Cipher Suite

Negotiated Connection:

```
Protocol : TLSv1.3
Cipher   : TLS_AES_256_GCM_SHA384
```

Cloudflare uses the same cipher suite as Google, demonstrating industry consensus on cryptographic best practices. Both prioritize AES-256 over AES-128 for higher security margin.

3.2.2 Certificate Chain

```
Certificate chain
0 s:C=US, ST=California, L=San Francisco, O=Cloudflare Inc.,
  CN=www.cloudflare.com
  i:C=US, O=Cloudflare Inc., CN=Cloudflare Inc RSA CA-2
1 s:C=US, O=Cloudflare Inc., CN=Cloudflare Inc RSA CA-2
  i:C=IE, O=Baltimore, OU=CyberTrust, CN=Baltimore CyberTrust Root

Verify return code: 0 (ok)
```

Notable Features:

- Shorter chain (2 vs. 3 certificates) - faster handshake
- Uses Baltimore CyberTrust Root (acquired by DigiCert)
- Cloudflare operates its own CA for edge certificate management

3.2.3 TLS Extensions

Extension	Status
ALPN (h2, http/1.1)	Supported
OCSP Stapling	Supported
Session Tickets	Supported

Cloudflare demonstrates complete extension support, including verified OCSP stapling for privacy-preserving certificate validation.

3.3 github.com

3.3.1 Protocol and Cipher Suite

```
Protocol : TLSv1.3
Cipher   : TLS_AES_128_GCM_SHA256
```

Cipher Suite Difference:

GitHub uses **AES-128-GCM** instead of AES-256-GCM:

- **Performance:** AES-128 is faster (10 rounds vs. 14 for AES-256)

- **Security:** 128-bit security ($2^{128} \approx 3.4 \times 10^{38}$ operations) is considered secure until at least 2030
- **Trade-off:** Prioritizes speed over theoretical security margin
- **Compliance:** Meets NIST recommendations for most applications

This is a reasonable engineering choice for high-throughput services.

3.3.2 Certificate Chain

```
Certificate chain
0 s:C=US, ST=California, L=San Francisco, O=GitHub Inc.,
  CN=github.com
i:C=US, O=DigiCert Inc, CN=DigiCert TLS Hybrid ECC SHA384 2020 CA1
1 s:C=US, O=DigiCert Inc, CN=DigiCert TLS Hybrid ECC SHA384 2020 CA1
i:C=US, O=DigiCert Inc, OU=www.digicert.com,
  CN=DigiCert Global Root CA

Verify return code: 0 (ok)
```

ECDSA Certificate Advantages:

- **Smaller size:** 256-bit ECC $\approx$ 3072-bit RSA security
- **Faster operations:** Signature verification is computationally cheaper
- **Bandwidth:** Smaller certificates reduce handshake overhead
- **Mobile-friendly:** Better for resource-constrained devices

The "Hybrid ECC" designation indicates support for both ECDSA and RSA signatures for compatibility.

4 SSL Labs Analysis Results

SSL Labs provides comprehensive security assessment with letter grades (A+ to F) based on four categories.

4.1 Overall Scores

Server	Overall	Cert	Protocol	Key Ex.	Cipher
www.google.com	A+	100	100	90	90
www.cloudflare.com	A+	100	100	90	90
github.com	A+	100	100	100	90

Table 3: SSL Labs security scores

4.2 Score Interpretation

Certificate (100/100):

- Valid chain to trusted root CA
- Strong key size: RSA $\geq$ 2048-bit or ECDSA $\geq$ 256-bit
- Modern signature algorithm: SHA-256 or better
- Appropriate validity period (typically 90 days for automated renewal)

Protocol Support (100/100):

- TLS 1.3 and 1.2 enabled
- All deprecated protocols disabled (SSL 3.0, TLS 1.0, TLS 1.1)
- No protocol downgrade vulnerabilities

Key Exchange (90-100):

- **100/100 (GitHub):** Only strong ECDHE with secure curves
- **90/100 (Google, Cloudflare):** May support additional curves for compatibility
- All configurations provide forward secrecy

Cipher Strength (90/100):

- Strong ciphers: AES-256-GCM, AES-128-GCM, ChaCha20-Poly1305
- No weak ciphers: RC4, DES, 3DES disabled
- Minor deduction likely due to cipher suite ordering or compatibility options

4.3 Vulnerability Testing

SSL Labs actively tests for known vulnerabilities:

Vulnerability	Description
BEAST	Browser Exploit Against SSL/TLS (CVE-2011-3389)
POODLE	Padding Oracle On Downgraded Legacy Encryption
Heartbleed	OpenSSL buffer over-read (CVE-2014-0160)
FREAK	Factoring RSA Export Keys
Logjam	Weak Diffie-Hellman parameters
DROWN	Decrypting RSA with Obsolete and Weakened eNcryption

Table 4: Tested vulnerability categories

Result: All three servers passed all vulnerability tests, confirming no known weaknesses in their TLS configurations.

5 Comparative Analysis

5.1 Methodological Differences

Aspect	OpenSSL	SSL Labs
Test Scope	Single connection	Multiple connections with various parameters
Protocol Testing	One version at a time	All versions simultaneously
Cipher Coverage	Negotiated suite only	All supported suites
Client Simulation	OpenSSL client only	Multiple browsers/OS combinations
Vulnerability Check	None	20+ known attacks
Result Format	Raw protocol data	Graded assessment (A+ to F)

Table 5: Comparison of analysis methodologies

5.2 Key Discrepancies and Implications

5.2.1 1. Test Coverage Difference

OpenSSL Behavior:

When connecting to a server, OpenSSL negotiates the *strongest mutually supported* option. For example:

```
# OpenSSL with modern capabilities connects with TLS 1.3
Protocol: TLSv1.3
Cipher: TLS_AES_256_GCM_SHA384
```

This doesn't reveal if the server also supports weaker options for legacy clients.

SSL Labs Behavior:

SSL Labs systematically tests *all* protocol versions and cipher suites, revealing:

- Complete list of 10-30 supported cipher suites
- Cipher preference order
- Weak ciphers enabled for compatibility
- Protocol version fallback behavior

Implication: A server may appear secure in OpenSSL testing but support vulnerable configurations detected by SSL Labs. This is why comprehensive testing is essential.

5.2.2 2. OCSP Stapling Detection

OpenSSL Test:

```
openssl s_client -connect www.google.com:443 \
  -servername www.google.com -status
```

Observed Results:

- www.google.com: OCSP response not always included
- www.cloudflare.com: OCSP response stapled
- github.com: OCSP response stapled

SSL Labs Results: All three servers support OCSP stapling.

Explanation: Server OCSP stapling may be conditional (e.g., based on connection timing, cache status). SSL Labs performs multiple tests to reliably confirm support. The `-status` flag requests stapling but doesn't guarantee the server will provide it.

5.2.3 3. Forward Secrecy Granularity

OpenSSL Assessment:

- TLS 1.3 connections *always* have forward secrecy (mandatory ECDHE)
- Binary result: FS present or absent

SSL Labs Assessment:

- Level 1: Some cipher suites support FS
- Level 2: Most cipher suites support FS
- Level 3: All cipher suites support FS
- Level 4: All cipher suites with modern key exchange (ECDHE)

This granularity identifies servers that support FS for modern clients but allow non-FS fallback for legacy systems.

5.2.4 4. Vulnerability Detection Gap

OpenSSL Limitations: OpenSSL shows *what is*, not *what's vulnerable*. It reports:

```
Protocol: TLSv1.2
Cipher: ECDHE-RSA-AES128-GCM-SHA256
```

But doesn't test for:

- Weak DH parameters (Logjam)
- SSL 3.0 POODLE vulnerability

- Export cipher support (FREAK)
- Compression-based attacks (CRIME, BREACH)

SSL Labs Active Testing:

- Attempts SSL 3.0 connection (should fail)
- Tests with weak DH parameters
- Checks for export cipher support
- Validates certificate revocation mechanisms

5.3 Security Posture Summary

Security Feature	Google	Cloudflare	GitHub
TLS 1.3 Support			
Legacy Protocols Disabled			
Forward Secrecy			
Strong Ciphers Only			
Valid Certificate Chain			
ALPN (HTTP/2)			
OCSF Stapling			
No Known Vulnerabilities			

Table 6: Comprehensive security feature comparison

All three servers demonstrate exemplary TLS configurations representing industry best practices.

5.4 Practical Implications

5.4.1 Tool Selection Guidelines

Use OpenSSL when:

- Debugging specific TLS handshake failures
- Verifying server responds to specific TLS version
- Quick connectivity testing during development
- Extracting certificate details for inspection
- Testing from specific network location (firewall rules, geo-blocking)

Use SSL Labs when:

- Performing comprehensive security audit
- Validating compliance (PCI DSS, HIPAA, etc.)
- Comparing against industry benchmarks
- Detecting configuration vulnerabilities
- Obtaining stakeholder-friendly security grade

Best Practice: Combine both approaches - OpenSSL for operational testing, SSL Labs for periodic comprehensive assessment.

5.4.2 Configuration Recommendations

Based on this analysis, a secure TLS configuration should:

1. Protocol Versions:

- Enable: TLS 1.3 (preferred), TLS 1.2 (compatibility)
- Disable: SSL 3.0, TLS 1.0, TLS 1.1

2. Cipher Suites:

- TLS 1.3: TLS_AES_256_GCM_SHA384, TLS_AES_128_GCM_SHA256, TLS_CHACHA20_POLY1305_SHA256
- TLS 1.2: ECDHE-RSA-AES256-GCM-SHA384, ECDHE-RSA-AES128-GCM-SHA256
- Disable: All CBC mode ciphers, RC4, DES, 3DES

3. Key Exchange:

- Require ECDHE for forward secrecy
- Use strong curves: X25519, P-256
- Disable RSA key exchange (no forward secrecy)

4. Certificates:

- RSA:
geq 2048 bits (3072+ recommended)
- ECDSA:
geq 256 bits (P-256 or P-384)
- Signature: SHA-256 or better
- Validity: 90 days (supports automated renewal)

5. Extensions:

- Enable ALPN for HTTP/2 negotiation
- Enable OCSP stapling for privacy
- Enable session tickets for performance
- Require SNI for virtual hosting

6 Conclusion

This homework analyzed TLS configurations of three high-profile web services (www.google.com, www.cloudflare.com, github.com) using complementary methodologies: protocol-level inspection with OpenSSL and policy-based evaluation with SSL Labs.

6.1 Key Findings

1. Excellent Industry Standards:

All analyzed servers demonstrate A+ security configurations:

- Modern protocols only (TLS 1.3, 1.2)
- Strong authenticated encryption (AES-GCM, ChaCha20-Poly1305)
- Perfect forward secrecy through ephemeral key exchange
- Valid certificate chains with proper validation
- Complete extension support (ALPN, OCSP stapling)
- Zero known vulnerabilities

2. Complementary Methodologies:

OpenSSL and SSL Labs serve different but equally important purposes:

- **OpenSSL:** Shows real-world connection behavior, useful for debugging
- **SSL Labs:** Reveals complete security posture, essential for auditing
- **Discrepancies:** Reflect scope differences (single test vs. comprehensive), not errors
- **Combined Use:** Provides complete security visibility

3. Industry Convergence:

Major technology companies have standardized on similar best practices:

- Cipher suite preferences: AES-256-GCM or AES-128-GCM with TLS 1.3
- Certificate strategies: Self-operated CAs (Google, Cloudflare) or commercial (GitHub)
- Extension adoption: Universal ALPN and OCSP stapling support
- Security-first approach: Compatibility sacrificed for deprecated protocols

6.2 Lessons Learned

Protocol-Level Analysis (OpenSSL):

- Provides direct observation of TLS handshakes
- Shows negotiated parameters in real-world scenarios
- Essential for debugging connection issues
- Limited to tested configurations only

Policy-Based Evaluation (SSL Labs):

- Comprehensive security assessment with grading
- Tests all supported configurations systematically
- Detects vulnerabilities through active testing
- Applies industry best practices and standards

6.3 Final Remarks

The combination of protocol-level inspection and policy-based evaluation provides complete TLS security visibility. While OpenSSL shows how a server behaves with specific clients, SSL Labs reveals the server's complete security posture and potential vulnerabilities.

Organizations deploying TLS should:

1. Use OpenSSL for operational testing and debugging
2. Use SSL Labs for quarterly comprehensive security audits
3. Follow industry leaders' example by disabling legacy protocols
4. Prioritize forward secrecy and authenticated encryption
5. Monitor for new vulnerabilities and update configurations accordingly

The analyzed servers demonstrate that strong security and broad compatibility are achievable through proper implementation of modern TLS protocols.